

Table of contents

Série 2	1
Méthodes	1
Méthode 1 : Analyse Syntaxique	1
Méthode 2 : Sémantique Opérationnelle	1
Méthode 3 : Spécification Formelle avec Logique du Premier Ordre (FOL)	1
Méthode 4 : Modélisation de Structures de Données (Tableaux)	2
Rappel	2
Exemples Illustratifs	2
Exemple 1 : Analyse Syntaxique et Contextuelle	2
Exemple 2 : Sémantique Opérationnelle avec Valuation et Mémoire	3
Exemple 3 : Spécification en FOL et Modélisation avec Tableaux	3
Synthèse	4

Série 2

Méthodes

Méthode 1 : Analyse Syntaxique

- **Vérification de la grammaire hors-contexte (CFG)** : identification des erreurs de structure de base du langage (parenthèses, ponctuation, mots-clés, ordre des tokens).
- **Vérification contextuelle** : identification des erreurs qui dépendent du contexte (variables non déclarées, types incompatibles, règles de portée).

Méthode 2 : Sémantique Opérationnelle

- **Évaluation par règles de transition** : utilisation de fonctions de valuation (**Val**) et de mémoire (**Mem**) pour décrire l'exécution pas à pas d'un programme.
- **Évaluation des expressions** : calcul de la valeur d'une expression dans un état mémoire donné.
- **Mise à jour de la mémoire** : modification de l'état mémoire lors de l'affectation ou d'autres actions.

Méthode 3 : Spécification Formelle avec Logique du Premier Ordre (FOL)

- **Préconditions** : conditions sur les entrées qui doivent être vraies avant l'exécution.
- **Postconditions** : conditions sur les sorties/mémoire qui doivent être vraies après l'exécution.

- **Formalisation de propriétés** : utilisation de quantificateurs, opérateurs logiques et arithmétiques pour exprimer des invariants, des buts.

Méthode 4 : Modélisation de Structures de Données (Tableaux)

- **Représentation des tableaux** : utilisation de `a.length` et `a[i]` pour manipuler des tableaux.
- **Parcours de tableau** : itération sur les indices avec des boucles `while` ou `for`.
- **Spécification de transformations** : description informelle puis formalisation en FOL.

Rappel

- **Distinction syntaxe/sémantique** : la syntaxe concerne la forme, la sémantique concerne le sens.
- **Sémantique opérationnelle** :
 - `Val(e, )` donne la valeur de l'expression `e` dans l'état mémoire
 - `Mem(P, )` donne l'état mémoire après l'exécution du programme `P` à partir de `.`
- **Lois de la logique** :
 - Élimination de l'implication : $(P \rightarrow Q) \equiv (\neg P \vee Q)$
 - Distributivité : $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$, etc.
 - De Morgan : $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$, $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
- **Préconditions/postconditions** : la correction partielle (si la précondition est vraie et que le programme termine, alors la postcondition est vraie) ; la terminaison doit être traitée à part.
- **Invariants de boucle** : propriété qui reste vraie avant et après chaque itération, utilisée pour prouver la correction des boucles.

Exemples Illustratifs

Exemple 1 : Analyse Syntaxique et Contextuelle

Contexte : Exercice 1 – détection d'erreurs dans un programme `remainder`.

Application :

- Le programme donné est :

```
remainder(a,b) = aux := b; while (res >= aux) res := res - aux; done res;
```

- **Erreur de grammaire hors-contexte** : le point-virgule après `res := res - aux` est incorrect selon la syntaxe décrite (probablement un séparateur d'instructions mal placé). De plus, la boucle `while` devrait avoir un bloc d'instructions délimité (par exemple avec `do ... done`), mais ici il manque peut-être un `do` après la condition.
- **Erreur contextuelle** : la variable `res` est utilisée mais n'a jamais été déclarée ou initialisée. De même, `aux` est déclarée mais `res` ne l'est pas. Cela viole une règle contextuelle (toute variable doit être déclarée avant utilisation).

Exemple 2 : Sémantique Opérationnelle avec Valuation et Mémoire

Contexte : Exercice 2 – évaluation de `abs(a,b)` dans l'état $\sigma = \{a \mapsto 3, b \mapsto 9\}$.

Application :

- On suppose que `abs(a,b)` est défini comme :
`abs(a,b) = res := 0; if (a < b) then res := b - a else res := a - b endif; res`
- Étape par étape :
 1. `Val(abs(3,9),  $\sigma$ ) = Val(programme de abs; res,  $\sigma$ )`
 2. On évalue le programme : `Mem(programme de abs,  $\sigma$ )`
 3. `Mem(res := 0,  $\sigma$ ) = [res  $\mapsto$  Val(0,  $\sigma$ )] =  $\sigma' = \{a \mapsto 3, b \mapsto 9, res \mapsto 0\}$`
 4. On évalue le `if` : `Val(a < b,  $\sigma'$ ) = (Val(a,  $\sigma'$ ) < Val(b,  $\sigma')) = (3 < 9) = \text{True}$`
 5. Donc on exécute `res := b - a` : `Mem(res := b - a,  $\sigma'$ ) =  $\sigma'' = \{a \mapsto 3, b \mapsto 9, res \mapsto 6\}$`
 6. Ensuite, on évalue `res` dans σ'' : `Val(res,  $\sigma''$ ) = 6`
- Ainsi, `Val(abs(3,9),  $\sigma$ ) = 6`.

Exemple 3 : Spécification en FOL et Modélisation avec Tableaux

Contexte : Exercice 3 – fonction qui remplit `b` avec les sommes cumulatives de `a`.

Application :

1. **Implémentation informelle** :

```

sum_prefix(a, b) =
  i := 0;
  sum := 0;
  while (i < a.length) do
    sum := sum + a[i];
    b[i] := sum;
    i := i + 1;
  done

```

2. **Précondition** : $a.length = b.length \quad a.length > 0$

3. **Postcondition** : $\forall j \in [0, a.length-1], b[j] = \sum_{k=0}^j a[k]$ En FOL, on pourrait écrire :

$$\forall j (0 \leq j < a.length \rightarrow b[j] = \text{sum}_{k=0}^j a[k])$$

où **sum** est une métanotation pour la somme (en FOL pur, on utilise une récurrence ou une fonction de sommation définie).

4. **Évaluation sémantique** pour $\sigma = \{a \leftarrow [0,3,5,6], b \leftarrow [0,0,0,0]\}$:

- Initialement $i=0, \text{sum}=0$.
- Itération 1 : $\text{sum}=0+0=0, b[0]=0, i=1$
- Itération 2 : $\text{sum}=0+3=3, b[1]=3, i=2$
- Itération 3 : $\text{sum}=3+5=8, b[2]=8, i=3$
- Itération 4 : $\text{sum}=8+6=14, b[3]=14, i=4$
- L'état final : $\sigma' = \{a \leftarrow [0,3,5,6], b \leftarrow [0,3,8,14], i \leftarrow 4, \text{sum} \leftarrow 14\}$

Synthèse

Les exercices mettent en œuvre plusieurs méthodes clés en vérification de logiciels : l'analyse syntaxique (grammaire et contexte), la sémantique opérationnelle (évaluation pas à pas avec états mémoire), et la modélisation formelle avec logique du premier ordre et structures de données (tableaux). La maîtrise de ces méthodes permet de détecter des erreurs, de spécifier précisément le comportement attendu et de raisonner sur l'exécution des programmes.